

AWS Invoice Processing Pipeline: Deployment and Lessons Learned

Phase 1: AWS Infrastructure Setup

S3 Bucket Creation

Two S3 buckets were created in the us-east-1 region to serve as the foundation of the invoice processing workflow:

- invoice-source-bucket-482784 (Created: 12/28/2025, 4:04:34 PM)
- This bucket receives raw invoice text files uploaded by users.
- It acts as the trigger point for the entire processing pipeline.

	name	AWS region	creation date
☐	invoice-source-bucket-482784	US East (N. Virginia) us-east-1	December 28, 2025, 16:04:34 (UTC-06:00)
☐	invoice-target-bucket-198329	US East (N. Virginia) us-east-1	December 28, 2025, 16:06:12 (UTC-06:00)

invoice-target-bucket-198329 (Created: 12/28/2025, 4:06:12 PM)

This bucket stores processed CSV files.

It serves as the final destination for transformed invoice data.

RDS MySQL Database Setup

A secure MySQL database was provisioned with the following configuration:

- Security Group: invoice-db-sg
- Allows MySQL traffic (port 3306) from the EC2 instance only.
- Ensures the database is accessible exclusively from the application server.

Database Instance:

Name: invoice-database

Engine: MySQL Community 8.0.43

Instance class: db.m7g.large (production-ready)

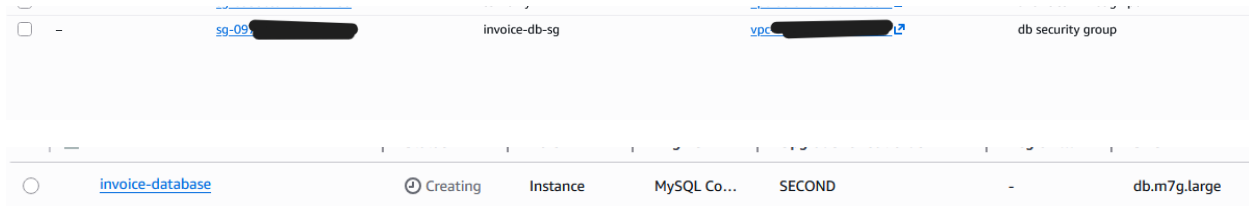
Endpoint: invoice-database.cq*****.us-east-1.rds.amazonaws.com

Database name: invoicedb

Master username: admin

Password: (hardcoded in views_final.py:15)

Purpose: Stores invoice metadata (Customer ID and Invoice ID) for quick lookups and analytics.



EC2 Instance Deployment

An EC2 instance was launched to host the Django application, with supporting security and IAM configurations:

- Security Group: invoice-ec2-sg
- Allows HTTP traffic on port 8080 (for Django server).
- Allows SSH on port 22 (for administration).

IAM Role: invoice-processor-role

Attached to the EC2 instance to grant necessary permissions.

Permissions include S3 read/write access, SNS publish/subscribe, and RDS connection.

EC2 Instance Details:

Instance ID: i-05be3*****

Type: t2.micro (free tier eligible)

Status: Running (initializing)

Availability Zone: us-east-1b

Public IP: 52.87.***.***

OS: Ubuntu Linux

Private IP: ip-172-31-***.***

☐	invoice-proces...	i-05be3[REDACTED]	Running	t2.micro	Initializing	View alarms +	us-east-1b
☐	-	sg-0d89[REDACTED]		invoice-ec2-sg		vpc-0cf6[REDACTED]	

SNS Topic Setup

An SNS topic was created to facilitate communication between S3 and the Django application:

- Name: invoice-upload-topic
- ARN: arn:aws:sns:us-east-1:<acct id>:invoice-upload-topic
- Protocol: HTTP
- Endpoint: http://52.87.***.***:8080/sns
- Purpose: Serves as a message broker; when files are uploaded to S3, messages are published to this SNS topic, notifying the Django application.

S3 Event Notification Configuration

S3 event notifications were set up to automate file processing:

- Event Name: invoice-upload-event
- Event Type: All object create events (s3:ObjectCreated:*)
- Triggers: PUT, POST, COPY operations, and multipart uploads
- Destination: SNS topic (invoice-upload-topic)
- Workflow: S3 Upload → S3 Event → SNS Topic → HTTP Endpoint (Django)

Event types

Specify at least one event for which you want to receive notifications

Object creation

- ☒ All object create events
s3:ObjectCreated:*

Phase 2: Application Deployment

SSH Connection to EC2

Established a secure SSH connection to the EC2 instance using the private key (invoice-key.pem). Logged in as the 'ubuntu' user, ready to deploy the Django application.

```
_InsightPipeline$ ssh -i invoice-key.pem ubuntu@52.87.153.173
```

Django Virtual Environment Activation

Navigated to the /opt/docproc/ directory and activated the Python virtual environment named django_env. The prompt changed to indicate successful activation. The environment is isolated, containing Django 6.0, boto3, and mysql-connector-python.

```
ubuntu@ip-172-31-153.173:/opt/docproc$ source ~/django_env/bin/activate
django_env) ubuntu@ip-172-31-153.173:/opt/docproc$
```

File Upload via SCP

Used Secure Copy Protocol (SCP) to transfer files from the local machine to the EC2 instance:

- Uploaded the entire docproc directory.
- Uploaded the api directory.
- Uploaded manage.py.
- Transferred key Django application files:
 - settings.py (Django configuration)
 - urls.py (URL routing)
 - wsgi.py (WSGI configuration)
 - views.py (main application logic)
 - manage.py (Django management script)
 - Other supporting Django app files

```
_InsightPipeline$ scp -i invoice-key.pem -r docproc/ ubuntu@52.87.153.173:/opt/docproc/
100%
100%
100%
100%
InsightPipeline$ scp -i invoice-key.pem -r api/ ubuntu@52.87.153.173:/opt/docproc/
```

Verify Deployment

Inspected the directory structure on EC2 under /opt/docproc/:

- api/ (Django app with invoice processing logic)
- django_env/ (Python virtual environment)
- docproc/ (main project configuration)
- manage.py (management script)

All necessary files were confirmed to be uploaded and ready for execution.

```
-31- [REDACTED]:/opt/docproc$ ls -la
ubuntu ubuntu 4096 Dec 28 22:56 .
root root 4096 Dec 28 22:46 ..
ubuntu ubuntu 4096 Dec 28 22:56 api
ubuntu ubuntu 4096 Dec 28 22:47 django_env
ubuntu ubuntu 4096 Dec 28 22:56 docproc
ubuntu ubuntu 805 Dec 28 22:56 manage.py
```

MySQL Database Initialization

Connected from EC2 to the RDS MySQL instance using the provided credentials. The database 'invoicedb' was created successfully and is ready to store invoice records. The application code also automatically creates the 'invoice' table via the create_table() function in views_final.py.

```
+-----+
| Database |
+-----+
| information_schema |
| invoicedb          |
| mysql              |
| performance_schema |
| sys                |
+-----+
5 rows in set (0.00 sec)
```

Phase 3: Application Testing

Start Django Development Server

Started the Django development server with `python manage.py runserver 0:8080`. The system performed checks and found no issues. The server was running on all interfaces (0.0.0.0) at port 8080, ready to receive HTTP requests from SNS.

SNS Subscription Confirmation

The SNS subscription to the Django endpoint was confirmed programmatically:

- Upon initial contact, SNS sent a `SubscriptionConfirmation` message to the application.
- The application detected the message type, extracted the `SubscribeURL`, and visited the URL to confirm the subscription.
- This logic was implemented in `api/views_final.py` (lines 30-37).

○ c634c783 [REDACTED]	http://52.87.173.146:8080/sns	✔ Confirmed	HTTP
-----------------------	-------------------------------	-------------	------

```
invoice-upload-topic.\\nTo confirm the subscription, visit the SubscribeURL included in this message.",\\n
```

Test Invoice Upload

A test file named `test_invoice.txt` (289 bytes) was uploaded to the source S3 bucket. The upload triggered the entire pipeline workflow.

□  test_invoice.txt	txt	December 28, 2025, 18:34:25 (UTC-06:00)	289.0 B	Standard
--	-----	---	---------	----------

Detailed Code Analysis

Main Application Logic (`api/views_final.py`)

SNS Message Handler (Lines 20-50)

The message function is exempt from CSRF, allowing SNS to POST messages without a token. It handles two message types:

- `SubscriptionConfirmation`: Confirms SNS subscriptions.
- `Notification`: Processes S3 event notifications and invokes document processing.

Returns HTTP 200 responses to acknowledge receipt.

Document Processing (Lines 83-114)

The `process_document` function parses SNS messages to extract S3 event details, downloads the file from S3, and replaces commas with semicolons in the content. It creates the database table if it doesn't exist and parses the invoice line by line for relevant fields. A critical bug was identified here: the `insert_data()` function was incorrectly called inside the parsing loop, resulting in multiple duplicate records per invoice file.

Database Operations (Lines 116-133)

The application creates the 'invoice' table and inserts only Customer ID and Invoice ID for each invoice, ensuring metadata storage in RDS while full invoice data remains in processed CSV files on S3.

CSV Generation (Lines 145-180)

The `transform_content` function parses the invoice for critical fields and formats them as a CSV string. Data sanitization is performed by replacing commas with semicolons to preserve CSV structure.

S3 Upload (Lines 135-143)

Processed CSV files are uploaded to the target S3 bucket. File names include a timestamp, Customer ID, and Invoice ID for uniqueness and traceability.

Complete Data Flow with Timings

The following sequence occurs when an invoice is uploaded:

1. User uploads `test_invoice.txt` to the source S3 bucket.
2. S3 detects the object creation and publishes an event to the SNS topic.
3. SNS sends an HTTP POST to the Django endpoint.
4. The Django application receives and routes the request.
5. The application parses the SNS message and extracts S3 event details.
6. The invoice file is downloaded from S3.
7. Connection to RDS MySQL is established and the table is created if necessary.
8. The invoice is parsed line by line for metadata.
9. A bug previously caused multiple duplicate database records per invoice line.
10. CSV is generated from the invoice data.

11. CSV is uploaded to the target S3 bucket.
12. The application returns HTTP 200 to SNS.

Bugs Fixed

Bug #1: Database Insert in Loop (Critical)

Previously, a database insert was performed for every line of the invoice, causing duplicate records and performance issues. The fix involved moving the insert operation outside the loop, reducing unnecessary writes and ensuring correct analytics.

Bug #2: Misleading TBD Comment

A comment indicating that CSV conversion was "to be done" was removed, as the functionality was already implemented, improving clarity for future developers.

Architecture Benefits

1. Serverless Event-Driven Architecture: Automatic processing upon file upload and scalable to thousands of invoices.
2. Decoupled Components: Independent S3, SNS, Django, and RDS/S3 components allow easy modifications and feature additions.
3. Data Durability: Triple redundancy with original invoices in S3, processed CSVs in S3, and metadata in RDS.
4. Cost Optimization: Utilizes AWS free tier, lightweight storage, and minimal costs for SNS and RDS.

Project Timeline

- 16:04: S3 source bucket created
- 16:06: S3 target bucket created
- 16:14: Database security group created
- 16:18: RDS MySQL instance created
- 16:25: EC2 security group created
- 16:25: IAM role created
- 16:25: EC2 instance launched
- 16:26: IAM role attached to EC2
- 16:31: SNS topic created
- 16:33: S3 event notifications configured
- 16:36: SSH connected to EC2

- 16:48: Django environment activated
- 16:56: Files uploaded via SCP
- 17:11: MySQL database initialized
- 18:21: Django server started
- 18:28: SNS subscription confirmed
- 18:34: Test invoice uploaded
- 19:05: Project documentation completed

Total time to completion: approximately 3 hours.

Lessons Learned

1. SNS subscription confirmation must be handled either programmatically or manually via AWS Console.
2. CSV data sanitization is essential; replacing commas with semicolons prevents corruption.
3. Database operations should be performed outside parsing loops to avoid logic errors.
4. Security groups must be configured to allow MySQL (3306) and HTTP (8080) traffic.
5. EC2 instances require appropriate IAM permissions for S3 and RDS access.